JMeter Performance Test Plan Report

1. Introduction

This document outlines the performance testing strategy using Apache JMeter for the CloveHR application. The purpose is to validate the system's behavior under load, identify bottlenecks, and ensure performance benchmarks are met.

2. Objective

- To assess the performance of the CloveHR application under expected and peak user loads.

- To determine the system's response time, throughput, resource utilization, and stability.

- To identify potential bottlenecks and areas for improvement.

3. Scope

In Scope

- Performance testing of the following endpoints/modules:

  - POST /api/Login

  - GET /api/users

  - GET /api/allhands

  - POST /api/apply_leave

  - POST /api/Submit_timesheet

- Post/api/Submit_TaxForm

- Load testing up to 1000 concurrent users.

- Monitoring response time, error rate, and throughput.

Out of Scope

- UI performance testing

- Security testing

- Testing of third-party integrations

By Prafull Mishra

4. Test Environment

Application URL: https://www.performance.clovehr.com/

Environment Type: Staging

JMeter Version: 5.5

Java Version: OpenJDK 17

OS: Windows 11 / Ubuntu 20.04

Monitors: Grafana, Prometheus, JMeter Dashboard

Database: MySQL 8.0

Web Server: Nginx

App Server: Tomcat

5. Test Strategy

Test Types

- Load Test

- Stress Test

- Spike Test

- Endurance Test

Test Scenarios

Scenario ID	Description	Endpoint	Method	Users
TS01	User login	/api/Login	POST	100
TS02	Fetch user list	/api/users	GET	200
TS03	View all-hands meetings	/api/allhands	GET	300
TS04	Apply for leave	/api/apply_leave	POST	250
TS05	Submit timesheet	/api/Submit_timesheet	POST	150
TS06	Submit tax Form	/api/Submit_taxForm	POST	100

By Prafull Mishra

6. Test Data Requirements

- 1000 test user accounts (pre-created in DB)

- Valid login credentials

- Relevant data for leave and timesheet submission

7. Tools & Scripts

- Apache JMeter 5.5

- JMeter Plugins Manager

- Grafana + Prometheus

- Shell/Bash Scripts for automation

8. Workload Model

| Load Level | Virtual Users | Duration | Ramp-up Time |

|---|---|---|---|

| Baseline | 100 | 10 mins | 1 min |

| Average Load | 500 | 15 mins | 3 mins |

| Peak Load | 1000 | 20 mins | 5 mins |

| Soak Test | 300 | 2 hours | 5 mins |

9. Entry & Exit Criteria

Entry Criteria

- Application is deployed and stable in staging.

- Test data is prepared.

- Monitoring tools are configured.

- Test scripts are reviewed.

By Prafull Mishra

Exit Criteria

- All planned tests are executed.

- Reports are generated and reviewed.

- No critical performance bottlenecks are found.

10. Performance Metrics

Metric	Description
Response Time	Time taken for API to return a response
Throughput	Number of requests processed per second
Error Rate	Percentage of failed requests
CPU/Memory Usage	Server resource usage during the test
Hits/sec	Number of hits on the server per second
Active Threads	Number of virtual users simulated

11. Risks & Mitigations

Risk	Mitigation
Environment instability	Use dedicated staging environment
Data unavailability	Prepare test data in advance
Network latency issues	Run tests from geographically close servers
Resource limits on client PC	Use distributed load testing with slaves

12. Test Execution Summary

Test Scenario	Max Response Time	Avg Response Time	Throughput (req/s)	Errors (%)
TS01 - Login	950 ms	320 ms	150	0.5%

By Prafull Mishra

| TS02 - Users | 1200 ms | 400 ms | 130 | 0.3% |

| TS03 - Allhands | 1400 ms | 500 ms | 120 | 0.6% |

| TS04 - Apply Leave | 1800 ms | 600 ms | 100 | 1.0% |

| TS05 - Timesheet | 2000 ms | 850 ms | 90 | 1.5% |

| TS06 – Tax Form | 2000 ms | 700 ms | 80 | 0.1% |

13. Analysis & Observations

- /api/Submit_timesheet endpoint showed high response times under load.

- Error rate exceeded 1% for TS05 scenario.

- CPU usage peaked at 85% on app servers.

- Memory utilization remained within limits.

14. Recommendations

- Optimize logic in /api/Submit_timesheet.

- Implement caching where applicable.

- Increase server capacity or thread pool settings.

- Optimize database queries.

15. Conclusion

The CloveHR application performs within acceptable limits under average and baseline loads. However, performance degrades under peak load for certain APIs. Recommended actions should be implemented before production deployment.

By Prafull Mishra